Bases de Données Avancées

Mon Cahier de Révisions

Université de Maroua
École Nationale Supérieure Polytechnique

Département INFOTEL - Niveau 4

Proposé par : Bounyamine Ousmanou

Année académique 2025-2026

CHAPITRE 1: XML et Technologies Associées

1.1 Introduction à XML

1.1.1 Définition et concepts de base

XML (eXtensible Markup Language) est un langage de balisage extensible conçu pour structurer, stocker et transporter des données. Contrairement à HTML qui se concentre sur la présentation, XML se concentre sur la description des données.

Caractéristiques principales

- **Extensible** : Vous pouvez créer vos propres balises
- **Auto-descriptif** : La structure décrit le contenu
- **Indépendant de la plateforme** : Peut être utilisé sur tout système
- **Standard ouvert** : Défini par le W3C

1.1.2 Syntaxe XML

Structure de base d'un document XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bibliotheque>
    <livre id="1">
        <titre>Introduction à XML</titre>
        <auteur>Jean Dupont</auteur>
        <annee>2024</annee>
        <prix devise="EUR">29.99</prix>
    </livre>
    <livre id="2">
        <titre>Bases de données NoSQL</titre>
        <auteur>Marie Martin</auteur>
        <annee>2023</annee>
        <prix devise="EUR">34.99</prix>
    </livre>
</bibliotheque>
```

Règles de syntaxe

1. **Document bien formé** : Tout document doit avoir une racine unique
2. **Balises fermées** : Toute balise ouverte doit être fermée
3. **Sensibilité à la casse** : `<Livre>` `<livre>`
4. **Imbrication correcte** : Les balises doivent être correctement imbriquées
5. **Valeurs d'attributs entre guillemets** : `id="1"`

1.2 XML Schema (XSD)

1.2.1 Introduction aux schémas

Un schéma XML définit la structure, le contenu et la sémantique d'un document XML. Il permet de valider qu'un document XML est conforme à une structure définie.

1.2.2 Types de données

Types de données XSD courants

- `xs:string` : Chaîne de caractères
- `xs:integer` : Nombre entier
- `xs:decimal` : Nombre décimal
- `xs:boolean` : Booléen (true/false)
- `xs:date` : Date (YYYY-MM-DD)
- `xs:time` : Heure (HH:MM:SS)

1.2.3 Exemple de schéma

Schéma XSD pour une bibliothèque

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

    <xs:element name="bibliotheque">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="livre" maxOccurs="unbounded">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="titre" type="xs:string"/>
                            <xs:element name="auteur" type="xs:string"/>
                            <xs:element name="annee" type="xs:gYear"/>
                            <xs:element name="prix">
                                <xs:complexType>
                                    <xs:simpleContent>
                                        <xs:extension base="xs:decimal">
                                            <xs:attribute name="devise"
                                                type="xs:string"/>
                                        </xs:extension>
                                    </xs:simpleContent>
                                </xs:complexType>
                            </xs:element>
                        </xs:sequence>
                        <xs:attribute name="id" type="xs:integer"
                            use="required"/>
                    </xs:complexType>
                </xs:element>
            </xs:sequence>
        </xs:complexType>
    </xs:element>

</xs:schema>
```

1.3 XPath

1.3.1 Introduction

XPath (XML Path Language) est un langage permettant de naviguer dans un document XML et de sélectionner des nœuds.

1.3.2 Axes principaux

Axe	Description
child	Sélectionne les enfants directs
descendant	Sélectionne tous les descendants
parent	Sélectionne le parent
ancestor	Sélectionne tous les ancêtres
attribute	Sélectionne les attributs
self	Sélectionne le nœud courant

Table 1: Axes XPath principaux

1.3.3 Exemples de requêtes

Requêtes XPath

```
// Tous les livres
//livre

// Titres de tous les livres
//livre/titre

// Livres avec id=1
//livre[@id='1']

// Prix des livres publiés après 2020
//livre[annee > 2020]/prix

// Nombre total de livres
count(//livre)

// Livre le plus récent
//livre[annee = max(//livre/annee)]

// Auteurs distincts
distinct-values(//livre/auteur)
```

1.4 XQuery

1.4.1 Introduction

XQuery est un langage de requête pour XML, similaire à SQL pour les bases de données relationnelles.

1.4.2 Structure FLWOR

XQuery utilise l'expression FLWOR (For, Let, Where, Order by, Return) :

- **For** : Itère sur une séquence
- **Let** : Définit une variable

- **Where** : Filtre les résultats
- **Order by** : Trie les résultats
- **Return** : Retourne les résultats

1.4.3 Exemples de requêtes

Requêtes XQuery

```
(: Livres triés par année :)
for $livre in //livre
order by $livre/annee descending
return $livre

(: Prix moyen des livres :)
let $prix := //livre/prix
return avg($prix)

(: Livres par auteur :)
for $auteur in distinct-values(//livre/auteur)
let $livres := //livre[auteur = $auteur]
return <auteur nom="{$auteur}">
         <nb_livres>{count($livres)}</nb_livres>
       </auteur>

(: Transformation en HTML :)
<html>
  <body>
    <h1>Catalogue</h1>
    <ul>
    {
      for $livre in //livre
      return <li>{$livre/titre/text()}</li>
    }
    </ul>
  </body>
</html>
```

1.5 XSLT

1.5.1 Introduction

XSLT (eXtensible Stylesheet Language Transformations) permet de transformer un document XML en un autre format (HTML, XML, texte).

1.5.2 Structure de base

Template XSLT de base

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="2.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

    <xsl:template match="/">
        <html>
            <head>
```

```
                <title>Bibliothèque</title>
            </head>
            <body>
                <h1>Liste des livres</h1>
                <table border="1">
                    <tr>
                        <th>Titre</th>
                        <th>Auteur</th>
                        <th>Année</th>
                        <th>Prix</th>
                    </tr>
                    <xsl:for-each select="//livre">
                        <tr>
                            <td><xsl:value-of select="titre"/></td>
                            <td><xsl:value-of select="auteur"/></td>
                            <td><xsl:value-of select="annee"/></td>
                            <td><xsl:value-of select="prix"/></td>
                        </tr>
                    </xsl:for-each>
                </table>
            </body>
        </html>
    </xsl:template>

</xsl:stylesheet>
```

CHAPITRE 2: MongoDB

2.1 Introduction à MongoDB

2.1.1 Qu'est-ce que MongoDB ?

MongoDB est une base de données NoSQL orientée documents. Elle stocke les données sous forme de documents JSON (BSON en interne) plutôt que dans des tables relationnelles.

2.1.2 Caractéristiques principales

- **Schéma flexible** : Pas de schéma rigide imposé
- **Scalabilité horizontale** : Facilite le sharding
- **Haute performance** : Optimisé pour la lecture/écriture
- **Réplication** : Support natif de la réplication
- **Requêtes riches** : Support de requêtes complexes

2.1.3 Concepts de base

2.2 Structure des documents

2.2.1 Format BSON

Les documents MongoDB sont stockés en BSON (Binary JSON), une extension binaire de JSON.

SQL	MongoDB
Base de données	Base de données
Table	Collection
Ligne	Document
Colonne	Champ
Index	Index

Table 2: Correspondance SQL/MongoDB

Exemple de document MongoDB

```
{
    "_id": ObjectId("507f1f77bcf86cd799439011"),
    "nom": "Dupont",
    "prenom": "Jean",
    "age": 30,
    "email": "jean.dupont@example.com",
    "adresse": {
        "rue": "123 Avenue des Champs",
        "ville": "Paris",
        "codePostal": "75008"
    },
    "hobbies": ["lecture", "sport", "musique"],
    "dateInscription": ISODate("2024-01-15T10:30:00Z")
}
```

2.3 Opérations CRUD

2.3.1 Création (Create)

Insertion de documents

```
// Insérer un document
db.utilisateurs.insertOne({
    nom: "Martin",
    prenom: "Marie",
    age: 25,
    email: "marie.martin@example.com"
})

// Insérer plusieurs documents
db.utilisateurs.insertMany([
    {nom: "Dubois", prenom: "Pierre", age: 35},
    {nom: "Bernard", prenom: "Sophie", age: 28}
])
```

2.3.2 Lecture (Read)

Requêtes de lecture

```
// Trouver tous les documents
db.utilisateurs.find()

// Trouver avec condition
db.utilisateurs.find({age: {$gte: 30}})

// Trouver un seul document
db.utilisateurs.findOne({nom: "Dupont"})

// Projection (sélection de champs)
db.utilisateurs.find({}, {nom: 1, prenom: 1, _id: 0})

// Tri
db.utilisateurs.find().sort({age: -1})

// Limitation
db.utilisateurs.find().limit(5)
```

2.3.3 Mise à jour (Update)

Opérations de mise à jour

```
// Mettre à jour un document
db.utilisateurs.updateOne(
    {nom: "Dupont"},
    {$set: {age: 31}}
)

// Mettre à jour plusieurs documents
db.utilisateurs.updateMany(
    {age: {$lt: 25}},
    {$inc: {age: 1}}
)

// Remplacer un document
db.utilisateurs.replaceOne(
    {nom: "Martin"},
    {nom: "Martin", prenom: "Marie", age: 26}
)

// Opérateurs de mise à jour courants
// $set : définir une valeur
// $inc : incrémenter
// $push : ajouter à un tableau
// $pull : retirer d'un tableau
// $unset : supprimer un champ
```

2.3.4 Suppression (Delete)

Opérations de suppression

```
// Supprimer un document
db.utilisateurs.deleteOne({nom: "Dubois"})

// Supprimer plusieurs documents
db.utilisateurs.deleteMany({age: {$lt: 18}})

// Supprimer tous les documents
db.utilisateurs.deleteMany({})
```

2.4 Agrégation

2.4.1 Pipeline d'agrégation

Le framework d'agrégation permet de traiter les données et retourner des résultats calculés.

Exemples d'agrégation

```
// Compter par catégorie
db.produits.aggregate([
    {$group: {
        _id: "$categorie",
        total: {$sum: 1}
    }}
])

// Moyenne des prix par catégorie
db.produits.aggregate([
    {$group: {
        _id: "$categorie",
        prixMoyen: {$avg: "$prix"}
    }},
    {$sort: {prixMoyen: -1}}
])

// Filtrer puis regrouper
db.ventes.aggregate([
    {$match: {date: {$gte: ISODate("2024-01-01")}}},
    {$group: {
        _id: "$produit",
        totalVentes: {$sum: "$quantite"},
        revenu: {$sum: {$multiply: ["$prix", "$quantite"]}}
    }},
    {$sort: {revenu: -1}},
    {$limit: 10}
])
```

2.5 Index

2.5.1 Types d'index

- **Index simple** : Sur un seul champ
- **Index composé** : Sur plusieurs champs

- **Index multiclé** : Sur des tableaux
- **Index de texte** : Pour la recherche textuelle
- **Index géospatial** : Pour les coordonnées

Création d'index

```
// Index simple
db.utilisateurs.createIndex({email: 1})

// Index composé
db.produits.createIndex({categorie: 1, prix: -1})

// Index unique
db.utilisateurs.createIndex({email: 1}, {unique: true})

// Index de texte
db.articles.createIndex({titre: "text", contenu: "text"})

// Lister les index
db.utilisateurs.getIndexes()

// Supprimer un index
db.utilisateurs.dropIndex("email_1")
```

2.6 Réplication

2.6.1 Replica Set

Un replica set est un groupe de serveurs MongoDB qui maintiennent le même ensemble de données.

Architecture

- **Primary** : Reçoit toutes les écritures
- **Secondary** : Réplique les données du primary
- **Arbiter** : Participe aux élections sans stocker de données

Configuration d'un replica set

```
// Initialiser le replica set
rs.initiate({
    _id: "monReplicaSet",
    members: [
        {_id: 0, host: "mongo1:27017"},
        {_id: 1, host: "mongo2:27017"},
        {_id: 2, host: "mongo3:27017"}
    ]
})

// Vérifier le statut
rs.status()

// Ajouter un membre
rs.add("mongo4:27017")

// Ajouter un arbitre
```

```
rs.addArb("mongo5:27017")
```

2.7 Sharding

2.7.1 Concept

Le sharding est une méthode de distribution des données sur plusieurs machines.

Composants

- **Shard** : Contient un sous-ensemble des données
- **Config Server** : Stocke les métadonnées
- **Query Router (mongos)** : Route les requêtes

Configuration du sharding

```
// Activer le sharding sur une base
sh.enableSharding("maBase")

// Sharder une collection
sh.shardCollection("maBase.maCollection", {_id: "hashed"})

// Vérifier le statut
sh.status()
```

CHAPITRE 3: Apache Cassandra

3.1 Introduction à Cassandra

3.1.1 Qu'est-ce que Cassandra ?

Apache Cassandra est une base de données NoSQL distribuée, conçue pour gérer de grandes quantités de données sur plusieurs serveurs sans point de défaillance unique.

3.1.2 Caractéristiques principales

- **Distribution** : Données réparties sur plusieurs nœuds
- **Scalabilité linéaire** : Performance proportionnelle au nombre de nœuds
- **Haute disponibilité** : Pas de point unique de défaillance
- **Tolérance aux pannes** : Réplication automatique
- **Performance d'écriture** : Optimisé pour les écritures massives

3.1.3 Architecture

Composants clés

- **Nœud** : Une instance Cassandra
- **Datacenter** : Collection de nœuds liés
- **Cluster** : Collection de datacenters
- **Commit log** : Journal des écritures
- **Memtable** : Structure en mémoire
- **SSTable** : Fichiers sur disque

3.2 Modélisation des données

3.2.1 Principes de modélisation

Contrairement aux bases relationnelles, Cassandra nécessite une modélisation orientée requêtes.

Règles de base

1. Modéliser en fonction des requêtes, pas des données
2. Minimiser les lectures de partitions
3. Dénormaliser autant que nécessaire
4. Une table par requête si nécessaire

3.2.2 Clés primaires

Structure de la clé primaire

- **Partition key** : Détermine la distribution des données
- **Clustering key** : Détermine l'ordre au sein d'une partition

```
PRIMARY KEY ((partition_key), clustering_key1, clustering_key2)
```

3.3 CQL (Cassandra Query Language)

3.3.1 Keyspace

Un keyspace est similaire à une base de données dans les SGBD relationnels.

Gestion des keyspaces

```cql
-- Créer un keyspace
CREATE KEYSPACE ecommerce
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 3
};

-- Utiliser un keyspace
USE ecommerce;
```

```cql
-- Lister les keyspaces
DESCRIBE KEYSPACES;

-- Supprimer un keyspace
DROP KEYSPACE ecommerce;
```

3.3.2 Tables

Création de tables

```cql
-- Table simple
CREATE TABLE utilisateurs (
    user_id UUID PRIMARY KEY,
    nom text,
    prenom text,
    email text,
    date_inscription timestamp
);

-- Table avec clé composée
CREATE TABLE commandes (
    user_id UUID,
    order_id UUID,
    date_commande timestamp,
    montant decimal,
    statut text,
    PRIMARY KEY ((user_id), order_id)
) WITH CLUSTERING ORDER BY (order_id DESC);

-- Table avec clustering multiple
CREATE TABLE produits_par_categorie (
    categorie text,
    sous_categorie text,
    produit_id UUID,
    nom text,
    prix decimal,
    PRIMARY KEY ((categorie), sous_categorie, produit_id)
);
```

3.3.3 Types de données

Type	Description
text, varchar	Chaîne de caractères
int, bigint	Nombres entiers
float, double	Nombres décimaux
decimal	Précision arbitraire
boolean	Booléen
timestamp	Date et heure
uuid, timeuuid	Identifiants uniques
list, set, map	Collections

Table 3: Types de données CQL

3.4 Opérations CRUD

3.4.1 Insertion

Insertion de données

```
-- Insertion simple
INSERT INTO utilisateurs (user_id, nom, prenom, email)
VALUES (uuid(), 'Dupont', 'Jean', 'jean@example.com');

-- Insertion avec TTL (Time To Live)
INSERT INTO sessions (session_id, user_id, data)
VALUES (uuid(), uuid(), 'session_data')
USING TTL 3600;

-- Insertion avec timestamp
INSERT INTO logs (log_id, message, timestamp)
VALUES (uuid(), 'User logged in', toTimestamp(now()))
USING TIMESTAMP 1234567890;
```

3.4.2 Sélection

Requêtes de sélection

```
-- Sélection simple
SELECT * FROM utilisateurs;

-- Avec condition sur la partition key
SELECT * FROM commandes WHERE user_id = uuid-value;

-- Avec condition sur clustering key
SELECT * FROM commandes
WHERE user_id = uuid-value
AND order_id > uuid-value;

-- Limitation
SELECT * FROM produits LIMIT 10;

-- Tri (uniquement sur clustering columns)
SELECT * FROM commandes
WHERE user_id = uuid-value
ORDER BY order_id DESC;

-- ALLOW FILTERING (à éviter en production)
SELECT * FROM utilisateurs
WHERE email = 'test@example.com' ALLOW FILTERING;
```

3.4.3 Mise à jour

Mises à jour

```
-- Mise à jour simple
UPDATE utilisateurs
SET email = 'nouveau@example.com'
WHERE user_id = uuid-value;
```

```
-- Mise à jour avec condition
UPDATE produits
SET prix = 29.99
WHERE produit_id = uuid-value
IF prix = 24.99;

-- Incrémenter un compteur
UPDATE compteurs
SET visites = visites + 1
WHERE page_id = 'accueil';
```

3.4.4 Suppression

Suppressions

```
-- Suppression complète
DELETE FROM utilisateurs WHERE user_id = uuid-value;

-- Suppression de colonnes
DELETE email FROM utilisateurs WHERE user_id = uuid-value;

-- Suppression avec condition
DELETE FROM produits
WHERE produit_id = uuid-value
IF prix < 10;
```

3.5 Collections et types définis par l'utilisateur

3.5.1 Collections

Utilisation des collections

```
-- Table avec collections
CREATE TABLE utilisateurs_avance (
    user_id UUID PRIMARY KEY,
    nom text,
    emails set<text>,
    telephones list<text>,
    preferences map<text, text>
);

-- Insertion avec collections
INSERT INTO utilisateurs_avance (user_id, nom, emails)
VALUES (uuid(), 'Martin', {'email1@example.com', 'email2@example.com'});

-- Ajouter à un set
UPDATE utilisateurs_avance
SET emails = emails + {'email3@example.com'}
WHERE user_id = uuid-value;

-- Ajouter à une list
UPDATE utilisateurs_avance
SET telephones = telephones + ['0123456789']
WHERE user_id = uuid-value;
```

```
-- Mettre à jour une map
UPDATE utilisateurs_avance
SET preferences['langue'] = 'fr'
WHERE user_id = uuid-value;
```

3.5.2 Types définis par l'utilisateur (UDT)

UDT

```
-- Créer un type
CREATE TYPE adresse (
    rue text,
    ville text,
    code_postal text,
    pays text
);

-- Utiliser le type dans une table
CREATE TABLE clients (
    client_id UUID PRIMARY KEY,
    nom text,
    adresse_livraison frozen<adresse>,
    adresse_facturation frozen<adresse>
);

-- Insertion avec UDT
INSERT INTO clients (client_id, nom, adresse_livraison)
VALUES (uuid(), 'Dupont', {
    rue: '123 Rue de la Paix',
    ville: 'Paris',
    code_postal: '75001',
    pays: 'France'
});
```

3.6 Index

Création d'index

```
-- Index simple
CREATE INDEX ON utilisateurs (email);

-- Index sur une collection
CREATE INDEX ON produits (values(tags));

-- Index personnalisé
CREATE CUSTOM INDEX ON articles (contenu)
USING 'org.apache.cassandra.index.sasi.SASIIndex';
```

CHAPITRE 4: Redis

4.1 Introduction à Redis

4.1.1 Qu'est-ce que Redis ?

Redis (REmote DIctionary Server) est une base de données en mémoire de type clé-valeur, connue pour ses performances exceptionnelles.

4.1.2 Caractéristiques principales

- **En mémoire** : Toutes les données en RAM
- **Structures de données avancées** : Plus que simple clé-valeur
- **Persistance optionnelle** : Snapshots ou AOF
- **Réplication** : Master-slave
- **Transactions** : Support ACID limité
- **Pub/Sub** : Messagerie publish-subscribe

4.2 Types de données

4.2.1 Strings

Opérations sur les chaînes

```
-- Définir une valeur
SET utilisateur:1:nom "Jean Dupont"

-- Récupérer une valeur
GET utilisateur:1:nom

-- Définir avec expiration (secondes)
SETEX session:abc123 3600 "session_data"

-- Définir si n'existe pas
SETNX compteur:visites 0

-- Incrémenter
INCR compteur:visites
INCRBY compteur:visites 10

-- Décrémenter
DECR compteur:stock
DECRBY compteur:stock 5

-- Manipulation de chaînes
APPEND utilisateur:1:nom " Jr."
STRLEN utilisateur:1:nom
GETRANGE utilisateur:1:nom 0 3
```

4.2.2 Listes

Opérations sur les listes

```
-- Ajouter à gauche/droite
LPUSH taches "tache1"
RPUSH taches "tache2" "tache3"

-- Récupérer et retirer
LPOP taches
RPOP taches

-- Obtenir une plage
LRANGE taches 0 -1

-- Longueur
LLEN taches

-- Récupérer par index
LINDEX taches 0

-- Définir par index
LSET taches 0 "nouvelle_tache"

-- Bloquer jusqu'à disponibilité
BLPOP taches 10
BRPOP taches 10
```

4.2.3 Sets

Opérations sur les ensembles

```
-- Ajouter des membres
SADD tags "redis" "nosql" "database"

-- Récupérer tous les membres
SMEMBERS tags

-- Vérifier l'appartenance
SISMEMBER tags "redis"

-- Nombre de membres
SCARD tags

-- Retirer des membres
SREM tags "nosql"

-- Membre aléatoire
SRANDMEMBER tags
SPOP tags

-- Opérations ensemblistes
SUNION set1 set2
SINTER set1 set2
SDIFF set1 set2
```

4.2.4 Hashes

Opérations sur les hashes

```
-- Définir un champ
HSET utilisateur:1 nom "Dupont"
HSET utilisateur:1 prenom "Jean"
HSET utilisateur:1 age "30"

-- Définir plusieurs champs
HMSET utilisateur:2 nom "Martin" prenom "Marie" age "25"

-- Récupérer un champ
HGET utilisateur:1 nom

-- Récupérer plusieurs champs
HMGET utilisateur:1 nom prenom

-- Récupérer tous les champs
HGETALL utilisateur:1

-- Vérifier l'existence
HEXISTS utilisateur:1 email

-- Supprimer un champ
HDEL utilisateur:1 age

-- Incrémenter
HINCRBY utilisateur:1 age 1

-- Obtenir toutes les clés/valeurs
HKEYS utilisateur:1
HVALS utilisateur:1

-- Nombre de champs
HLEN utilisateur:1
```

4.2.5 Sorted Sets

Ensembles triés

```
-- Ajouter avec score
ZADD classement 100 "joueur1"
ZADD classement 85 "joueur2" 95 "joueur3"

-- Obtenir par rang
ZRANGE classement 0 -1
ZRANGE classement 0 -1 WITHSCORES

-- Obtenir par score
ZRANGEBYSCORE classement 80 100

-- Rang d'un membre
ZRANK classement "joueur1"
ZREVRANK classement "joueur1"

-- Score d'un membre
ZSCORE classement "joueur1"
```

```
-- Incrémenter le score
ZINCRBY classement 5 "joueur2"

-- Supprimer
ZREM classement "joueur3"

-- Nombre de membres
ZCARD classement

-- Compter par plage de scores
ZCOUNT classement 80 100
```

4.3 Gestion des clés

Commandes générales

```
-- Vérifier l'existence
EXISTS utilisateur:1

-- Supprimer
DEL utilisateur:1

-- Expiration
EXPIRE session:123 3600
EXPIREAT session:123 1672531200

-- Temps restant
TTL session:123

-- Supprimer l'expiration
PERSIST session:123

-- Renommer
RENAME ancienne_cle nouvelle_cle

-- Type de donnée
TYPE utilisateur:1

-- Recherche de clés (attention en production!)
KEYS utilisateur:*
SCAN 0 MATCH utilisateur:* COUNT 100
```

4.4 Transactions

Transactions Redis

```
-- Transaction simple
MULTI
SET compte:1 100
INCR compteur:operations
EXEC

-- Avec surveillance (optimistic locking)
WATCH compte:1
```

```
MULTI
DECRBY compte:1 50
INCRBY compte:2 50
EXEC

-- Abandonner une transaction
MULTI
SET key1 "value1"
DISCARD
```

4.5 Pub/Sub

Publication/Souscription

```
-- S'abonner à un canal
SUBSCRIBE notifications

-- S'abonner avec pattern
PSUBSCRIBE news:*

-- Publier un message
PUBLISH notifications "Nouveau message"

-- Lister les canaux actifs
PUBSUB CHANNELS

-- Nombre d'abonnés
PUBSUB NUMSUB notifications

-- Se désabonner
UNSUBSCRIBE notifications
```

4.6 Persistance

4.6.1 Snapshots (RDB)

Configuration pour sauvegardes périodiques :

```
save 900 1      # Au moins 1 clé modifiée en 15 min
save 300 10     # Au moins 10 clés en 5 min
save 60 10000   # Au moins 10000 clés en 1 min
```

4.6.2 Append-Only File (AOF)

Configuration pour log des opérations :

```
appendonly yes
appendfsync everysec  # always, everysec, no
```

4.7 Réplication

Configuration Master-Slave

```
-- Sur le slave
REPLICAOF 192.168.1.100 6379

-- Vérifier le rôle
ROLE

-- Promouvoir un slave en master
REPLICAOF NO ONE

-- Informations sur la réplication
INFO replication
```

CHAPITRE 5: Neo4j

5.1 Introduction à Neo4j

5.1.1 Qu'est-ce que Neo4j ?

Neo4j est une base de données orientée graphes qui stocke les données sous forme de nœuds et de relations.

5.1.2 Concepts fondamentaux

- **Nœud (Node)** : Entité du graphe
- **Relation (Relationship)** : Connexion entre nœuds
- **Propriété (Property)** : Attribut d'un nœud ou d'une relation
- **Label** : Catégorie d'un nœud
- **Type** : Catégorie d'une relation

5.1.3 Quand utiliser Neo4j ?

- Réseaux sociaux
- Systèmes de recommandation
- Détection de fraude
- Gestion des connaissances
- Analyse de dépendances
- Gestion d'identités et d'accès

5.2 Cypher : Le langage de requête

5.2.1 Syntaxe de base

Cypher utilise des patterns ASCII art pour représenter les graphes :

- `()` : Nœud
- `->` ou `<-` : Relation directionnelle
- `-` : Relation non directionnelle
- `[]` : Propriétés de relation

5.3 Création de données

5.3.1 Créer des nœuds

Création de nœuds

```
// Nœud simple
CREATE (p:Personne)

// Nœud avec propriétés
CREATE (p:Personne {nom: 'Dupont', prenom: 'Jean', age: 30})

// Multiples labels
CREATE (p:Personne:Employé {nom: 'Martin'})

// Plusieurs nœuds
CREATE
  (p1:Personne {nom: 'Alice'}),
  (p2:Personne {nom: 'Bob'}),
  (p3:Personne {nom: 'Charlie'})
```

5.3.2 Créer des relations

Création de relations

```
// Créer des nœuds et relation
CREATE (p1:Personne {nom: 'Alice'})
-[:CONNAIT]->(p2:Personne {nom: 'Bob'})

// Relation avec propriétés
CREATE (p1:Personne {nom: 'Alice'})
-[:CONNAIT {depuis: 2020, type: 'ami'}]->
(p2:Personne {nom: 'Bob'})

// Créer relation entre nœuds existants
MATCH (a:Personne {nom: 'Alice'})
MATCH (b:Personne {nom: 'Charlie'})
CREATE (a)-[:CONNAIT]->(b)

// Ou avec MERGE
MATCH (a:Personne {nom: 'Alice'})
MATCH (b:Personne {nom: 'Charlie'})
MERGE (a)-[:CONNAIT]->(b)
```

5.4 Lecture de données

5.4.1 Patterns de base

Requêtes MATCH

```
// Tous les nœuds Personne
MATCH (p:Personne)
RETURN p

// Avec condition
MATCH (p:Personne)
WHERE p.age > 25
RETURN p.nom, p.age

// Relations
MATCH (p1:Personne)-[:CONNAIT]->(p2:Personne)
RETURN p1.nom, p2.nom

// Relations avec propriétés
MATCH (p1:Personne)-[r:CONNAIT]->(p2:Personne)
WHERE r.depuis > 2015
RETURN p1.nom, p2.nom, r.depuis

// Chemins de longueur variable
MATCH (p1:Personne)-[:CONNAIT*1..3]->(p2:Personne)
WHERE p1.nom = 'Alice'
RETURN p2.nom

// Chemins les plus courts
MATCH path = shortestPath(
  (p1:Personne {nom: 'Alice'})-[:CONNAIT*]->
  (p2:Personne {nom: 'David'})
)
RETURN path
```

5.4.2 Filtrage et tri

WHERE

```
// Filtrage multiple
MATCH (p:Personne)
WHERE p.age >= 25 AND p.age <= 35
RETURN p

// Avec OR
MATCH (p:Personne)
WHERE p.nom = 'Dupont' OR p.nom = 'Martin'
RETURN p

// Vérifier l'existence de propriété
MATCH (p:Personne)
WHERE p.email IS NOT NULL
RETURN p

// Pattern dans WHERE
MATCH (p:Personne)
WHERE (p)-[:TRAVAILLE_POUR]->(:Entreprise {nom: 'TechCorp'})
```

```
RETURN p

// Tri
MATCH (p:Personne)
RETURN p.nom, p.age
ORDER BY p.age DESC

// Limitation
MATCH (p:Personne)
RETURN p
LIMIT 10

// Pagination
MATCH (p:Personne)
RETURN p
ORDER BY p.nom
SKIP 20 LIMIT 10
```

5.5 Mise à jour de données

5.5.1 SET et REMOVE

Modification de propriétés

```
// Mettre à jour une propriété
MATCH (p:Personne {nom: 'Dupont'})
SET p.age = 31

// Ajouter plusieurs propriétés
MATCH (p:Personne {nom: 'Dupont'})
SET p.email = 'dupont@example.com', p.ville = 'Paris'

// Ajouter un label
MATCH (p:Personne {nom: 'Dupont'})
SET p:Manager

// Copier toutes les propriétés
MATCH (p1:Personne {nom: 'Alice'})
MATCH (p2:Personne {nom: 'Bob'})
SET p2 = p1

// Supprimer une propriété
MATCH (p:Personne {nom: 'Dupont'})
REMOVE p.email

// Supprimer un label
MATCH (p:Personne {nom: 'Dupont'})
REMOVE p:Manager
```

5.5.2 DELETE

Suppression

```
// Supprimer un nœud (doit être isolé)
MATCH (p:Personne {nom: 'Test'})
DELETE p

// Supprimer avec relations
MATCH (p:Personne {nom: 'Test'})
DETACH DELETE p

// Supprimer une relation
MATCH (p1:Personne)-[r:CONNAIT]->(p2:Personne)
WHERE p1.nom = 'Alice' AND p2.nom = 'Bob'
DELETE r

// Supprimer tous les nœuds d'un type
MATCH (p:Personne)
DETACH DELETE p
```

5.6 Agrégation et fonctions

5.6.1 Fonctions d'agrégation

Agrégation

```
// Compter
MATCH (p:Personne)
RETURN COUNT(p) AS nb_personnes

// Moyenne d'âge
MATCH (p:Personne)
RETURN AVG(p.age) AS age_moyen

// Min, Max, Sum
MATCH (p:Personne)
RETURN MIN(p.age), MAX(p.age), SUM(p.age)

// Collecter dans une liste
MATCH (p:Personne)
RETURN COLLECT(p.nom) AS noms

// Regrouper et compter
MATCH (p:Personne)
RETURN p.ville, COUNT(p) AS nb_habitants
ORDER BY nb_habitants DESC

// Relations par personne
MATCH (p:Personne)-[r:CONNAIT]->()
RETURN p.nom, COUNT(r) AS nb_connexions
ORDER BY nb_connexions DESC
```

5.6.2 Fonctions de liste et de chaîne

Fonctions utilitaires

```
// Fonctions de chaîne
MATCH (p:Personne)
RETURN UPPER(p.nom), LOWER(p.prenom)

MATCH (p:Personne)
RETURN SUBSTRING(p.nom, 0, 3)

// Fonctions de liste
WITH [1, 2, 3, 4, 5] AS nombres
RETURN SIZE(nombres), HEAD(nombres), TAIL(nombres)

// Filtrer une liste
WITH [1, 2, 3, 4, 5] AS nombres
RETURN [x IN nombres WHERE x > 2] AS filtres

// Transformation
WITH [1, 2, 3] AS nombres
RETURN [x IN nombres | x * 2] AS doubles
```

5.7 Index et contraintes

5.7.1 Index

Gestion des index

```
// Créer un index
CREATE INDEX FOR (p:Personne) ON (p.nom)

// Index composite
CREATE INDEX FOR (p:Personne) ON (p.nom, p.prenom)

// Index de texte intégral
CREATE FULLTEXT INDEX nomIndex
FOR (p:Personne) ON EACH [p.nom, p.prenom]

// Lister les index
SHOW INDEXES

// Supprimer un index
DROP INDEX index_name
```

5.7.2 Contraintes

Contraintes

```
// Contrainte d'unicité
CREATE CONSTRAINT FOR (p:Personne)
REQUIRE p.email IS UNIQUE

// Contrainte d'existence
CREATE CONSTRAINT FOR (p:Personne)
REQUIRE p.nom IS NOT NULL
```

```
// Contrainte de clé
CREATE CONSTRAINT FOR (p:Personne)
REQUIRE (p.id) IS NODE KEY

// Lister les contraintes
SHOW CONSTRAINTS

// Supprimer une contrainte
DROP CONSTRAINT constraint_name
```

5.8 Patterns avancés

5.8.1 OPTIONAL MATCH

Correspondance optionnelle

```
// Équivalent du LEFT JOIN
MATCH (p:Personne)
OPTIONAL MATCH (p)-[:TRAVAILLE_POUR]->(e:Entreprise)
RETURN p.nom, e.nom

// Avec condition
MATCH (p:Personne {nom: 'Alice'})
OPTIONAL MATCH (p)-[r:CONNAIT]->(ami:Personne)
WHERE ami.age > 30
RETURN p, COLLECT(ami.nom) AS amis_30plus
```

5.8.2 WITH et pipeline

Chaînage de requêtes

```
// Filtrer puis agréger
MATCH (p:Personne)-[:CONNAIT]->(ami:Personne)
WHERE p.age > 25
WITH p, COUNT(ami) AS nb_amis
WHERE nb_amis > 5
RETURN p.nom, nb_amis

// Calculs intermédiaires
MATCH (p:Personne)
WITH p, p.age * 365 AS jours_vie
WHERE jours_vie > 10000
RETURN p.nom, jours_vie
```

5.8.3 UNION

Combiner des résultats

```
// Union de deux requêtes
MATCH (p:Personne) WHERE p.age < 25
RETURN p.nom, p.age
UNION
MATCH (p:Personne) WHERE p.age > 60
RETURN p.nom, p.age

// UNION ALL (avec doublons)
MATCH (p:Personne) WHERE p.ville = 'Paris'
RETURN p.nom
UNION ALL
MATCH (p:Personne) WHERE p.ville = 'Lyon'
RETURN p.nom
```

5.9 Cas d'usage pratiques

5.9.1 Réseau social

Recommandation d'amis

```
// Amis d'amis (recommandations)
MATCH (moi:Personne {nom: 'Alice'})
-[:CONNAIT]->()-[:CONNAIT]->(recommandation)
WHERE NOT (moi)-[:CONNAIT]->(recommandation)
AND moi <> recommandation
RETURN recommandation.nom, COUNT(*) AS connexions_communes
ORDER BY connexions_communes DESC
LIMIT 5

// Influenceurs (plus connectés)
MATCH (p:Personne)-[:CONNAIT]->()
RETURN p.nom, COUNT(*) AS nb_suiveurs
ORDER BY nb_suiveurs DESC
LIMIT 10
```

5.9.2 Système de recommandation

Produits recommandés

```
// Produits achetés par des clients similaires
MATCH (moi:Client {id: 123})-[:A_ACHETE]->(p:Produit)
<-[:A_ACHETE]-(autre:Client)-[:A_ACHETE]->(recommandation)
WHERE NOT (moi)-[:A_ACHETE]->(recommandation)
RETURN recommandation.nom, COUNT(*) AS score
ORDER BY score DESC
LIMIT 5

// Produits similaires
MATCH (p:Produit {id: 456})-[:DE_CATEGORIE]->(c:Categorie)
<-[:DE_CATEGORIE]-(similaire:Produit)
WHERE p <> similaire
RETURN similaire.nom, similaire.prix
```

```
LIMIT 10
```

CHAPITRE A: Ressources et références

A.1 Documentation officielle

- XML : https://www.w3.org/XML/
- MongoDB : https://docs.mongodb.com/
- Cassandra : https://cassandra.apache.org/doc/
- Redis : https://redis.io/documentation
- Neo4j : https://neo4j.com/docs/